PÓS TECH

DEVOPS E ARQUITETURA CLOUD

FASE 4 | AULA 03 -

DATADOG - TRACING E LOGS EM LARGA ESCALA

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 5
O QUE VOCÊ VIU NESTA AULA? 40
REFERÊNCIAS 41

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O QUE VEM POR AÍ?

Nesta aula, você aprenderá os fundamentos essenciais do gerenciamento de logs em ambientes modernos de microsserviços e cloud. Serão apresentados os conceitos de logs estruturados, pipelines de processamento, correlação automática entre logs e traces, estratégias de indexação e mecanismos de controle de custo. O conteúdo irá te preparar para compreender não apenas como funcionam os componentes do Log Management do Datadog, mas também como projetar uma arquitetura observável, sustentável e alinhada às melhores práticas do mercado.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

HANDS ON

Nesta aula prática, vamos configurar logs estruturados em aplicações .NET utilizando o Serilog para a emissão de dados em formato JSON, facilitando a ingestão por plataformas de monitoramento. O foco inicial será habilitar a correlação automática entre logs e traces por meio do dd-trace-dotnet, permitindo uma análise detalhada dos vínculos diretamente na interface do Datadog. Além disso, desenvolveremos pipelines de processamento completos, abordando desde o uso de parsers e remapeamentos até a categorização e definição de status para as mensagens de log.

Na segunda etapa, o aprendizado focará na organização e otimização dos dados através da criação de facets para filtros rápidos e análises avançadas no Log Explorer. Serão exploradas técnicas de configuração de índices de log, abrangendo a definição de períodos de retenção e a aplicação de filtros de ingestão e exclusão para uma gestão eficiente de custos. Por fim, os alunos e alunas aplicarão estratégias de amostragem de logs e traces, compreendendo na prática como essas escolhas impactam tanto a capacidade de troubleshooting quanto o orçamento do projeto.

Ao final do exercício, você dominará a base para construir estratégias de logging escaláveis em produção.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

SAIBA MAIS

Arquitetura de Log Management do Datadog

Arquitetura de Log Management

A gestão de logs em ambientes de microsserviços e infraestruturas complexas representa um desafio significativo. O volume, a variedade e a velocidade dos logs gerados exigem uma plataforma robusta que não apenas armazene, mas também permita a análise e a correlação desses dados de forma eficiente. O Datadog aborda essa necessidade com uma arquitetura de Log Management projetada para ser escalável, flexível e profundamente integrada ao seu ecossistema de observabilidade.

A arquitetura de Log Management do Datadog é fundamentada no princípio de **Logging without Limits**. Este conceito representa uma mudança de paradigma em relação às soluções de logging tradicionais, que frequentemente forçam as organizações a tomar decisões difíceis sobre quais logs coletar e indexar, muitas vezes antes mesmo de compreenderem seu valor potencial. A principal inovação do Datadog é a **dissociação entre a ingestão e a indexação de logs**, permitindo que 100% dos logs gerados por uma aplicação ou infraestrutura sejam coletados e processados, enquanto apenas um subconjunto estrategicamente selecionado seja pago para ser indexado e retido para análise interativa.

Essa abordagem oferece o melhor de dois mundos: a visibilidade completa necessária para auditorias de segurança e investigações forenses, e o controle de custos essencial para a sustentabilidade financeira da operação. Todos os logs, indexados ou não, podem ser arquivados em um armazenamento de baixo custo (como Amazon S3, Google Cloud Storage ou Azure Blob Storage) e podem ser utilizados para gerar métricas em tempo real.

A arquitetura é composta por componentes interconectados que garantem um fluxo de dados contínuo e enriquecido, desde a origem até a visualização.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Componente	Descrição	Função Principal
Datadog Agent	Um software leve instalado em hosts, containers ou VMs.	Coleta logs de arquivos, `stdout` / `stderr`, sockets (TCP/UDP), e fontes de sistema como `journald` e Windows Event Channels. Adiciona metadados de infraestrutura aos logs.
Endpoints de Ingestão	Pontos de entrada seguros na plataforma Datadog para receber dados de log.	Recebem logs do Agent, de forwarders de nuvem (como AWS Lambda), ou diretamente de aplicações via API HTTP.
Pipelines de Processamento	Sequências de processadores que analisam, transformam e enriquecem os logs recebidos.	Parsing de formatos (JSON, Grok), extração de atributos, remapeamento de campos, e enriquecimento com dados de negócio ou geolocalização.
Indexes (Hot Storage)	Armazenamento de alta performance para logs que necessitam de análise rápida e interativa.	Permite buscas complexas, agregações, criação de monitores e dashboards sobre os dados de log recentes.
Archives (Cold Storage)	Armazenamento de baixo custo em provedores de nuvem (AWS S3, Azure Blob Storage, GCS).	Armazena todos os logs ingeridos por longos períodos para conformidade, auditoria e análise futura via reidratação.
Log Explorer	A interface de usuário para busca, visualização e análise de logs.	Oferece busca full-text, filtragem por facetas, visualização de logs em tempo real (Live Tail) e análise de padrões.

Figura 1 – Arquitetura de Log Management
Fonte: Elaborado pelo autor (2026)

Essa arquitetura distribuída e desacoplada permite que as equipes de engenharia e operações obtenham visibilidade completa de seus sistemas sem o receio de custos proibitivos, transformando logs de um fardo de armazenamento em um ativo estratégico para a observabilidade.

O fluxo de dados na arquitetura do Datadog pode ser resumido nas seguintes etapas:

Coleta: o Datadog Agent, instalado em hosts, como um DaemonSet em Kubernetes, ou através de integrações nativas com serviços de nuvem, coleta os logs diretamente da fonte. Ele pode fazer o “tail” de arquivos, capturar o stdout/stderr de containers, ou receber logs via TCP/UDP e outros protocolos.

Ingestão e Processamento: os logs são enviados para a plataforma Datadog, onde passam por **Pipelines de Processamento**. Nesta fase, os logs são parseados (por exemplo, extraindo atributos de um JSON), enriquecidos com metadados (como tags de infraestrutura e de negócio) e transformados. Essas operações são realizadas em todos os logs que são ingeridos, independentemente de serem ou não indexados.

Decisão de Roteamento: após o processamento, a plataforma decide o destino de cada log com base em regras de configuração:

- **Indexação**: logs que correspondem aos filtros de um **Índice** são retidos para busca, análise e visualização no Log Explorer. É nesta etapa que os

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

custos são primariamente incorridos. Os índices permitem definir diferentes períodos de retenção para diferentes tipos de logs.

- **Arquivamento**: todos os logs (ou um subconjunto definido) são encaminhados para um "cold storage" de longo prazo, garantindo conformidade e a possibilidade de análise futura.
- **Geração de Métricas**: métricas podem ser geradas a partir do fluxo de logs ingeridos, permitindo a criação de dashboards e monitores sobre tendências de alto volume sem a necessidade de indexar os logs brutos.

Análise e Alertas: os logs indexados estão disponíveis no **Log Explorer** para busca, agregação e visualização. Eles podem ser correlacionados com traces e métricas, e utilizados para criar monitores que alertam sobre anomalias ou condições específicas.

Reidratação: caso seja necessário investigar logs que não foram indexados, o mecanismo de **Rehydration** permite trazer de volta os logs arquivados para dentro do Datadog para uma análise temporária, sob demanda.

Esta arquitetura flexível permite que as equipes de engenharia ajustem dinamicamente suas estratégias de logging, otimizando a relação entre visibilidade e custo sem sacrificar a capacidade de responder a incidentes ou cumprir requisitos de conformidade.

Componente	Descrição	Implicações de Custo
Ingestão	Coleta e processamento de todos os logs.	Custo baixo, geralmente associado ao volume de dados transferidos.
Indexação	Armazenamento de logs para busca e análise interativa.	Principal fator de custo, baseado no volume de dados e no período de retenção.
Arquivamento	Armazenamento de longo prazo em storage do cliente.	Custo do provedor de nuvem (S_3, GCS, Azure), geralmente muito baixo.
Reidratação	Análise sob demanda de logs arquivados.	Custo por volume de dados escaneado e re-ingerido para análise.

Figura 2 – Estratégias de logging
Fonte: Elaborado pelo autor (2026)

O ciclo completo de vida de um log no Datadog

Para compreender plenamente o poder da plataforma de Log Management do Datadog, é essencial acompanhar a jornada de um log, desde sua criação na origem até sua eventual análise ou arquivamento. Este ciclo de vida é composto por uma

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

série de etapas bem definidas que garantem processamento eficiente, enriquecimento contextual e roteamento inteligente.

Geração (Generation): um evento ocorre em uma aplicação ou sistema. Uma aplicação .NET, por exemplo, utiliza uma biblioteca de logging como o Serilog para escrever uma mensagem de log. Neste ponto, a boa prática é gerar logs estruturados (em formato JSON), que já contêm pares chave-valor, como {"level": "Error", "message": "Failed to process payment", "orderId": "xyz-123"}.

Coleta (Collection): o Datadog Agent, que está em execução no mesmo host ou nó, detecta o novo log. A coleta pode ocorrer de várias formas: monitorando a saída padrão (stdout) de um container, fazendo “tail” de um arquivo de log (/app/logs/service.log), ou recebendo eventos do sistema operacional (como o Windows Event Log).

Envio (Shipping): o Agent envia o log de forma segura para o endpoint de ingestão do Datadog. Durante este processo, o Agent já adiciona metadados valiosos, como tags do host, tags de containers (ex: image_name, pod_name), e tags de integrações de nuvem (ex: availability-zone).

Processamento em Pipeline (Pipeline Processing): ao chegar na plataforma Datadog, o log entra em uma série de Pipelines de Processamento. Cada pipeline possui um filtro opcional para selecionar a quais logs ele se aplica. Dentro do pipeline, uma sequência de Processors é executada:

- **Parsing:** se o log não estiver em JSON, um processador Grok pode ser usado para extrair atributos de texto não estruturado.
- **Enriquecimento (Enrichment):** um processador de remapeamento (Attribute Remapper) pode renomear campos para padronização (ex: level para status). Um processador de categoria pode atribuir um nível de importância com base no conteúdo.
- **Processamento Geográfico:** um processador pode extrair a localização geográfica a partir de um endereço IP.

Roteamento (Routing): após o processamento, o log enriquecido chega a uma bifurcação crítica onde seu destino é decidido:

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **Arquivamento (Archiving):** com base nas configurações de arquivamento, uma cópia do log é enviada para um sistema de armazenamento de longo prazo, como um bucket Amazon S3. Esta etapa é independente da indexação e é recomendada para todos os logs, garantindo uma cópia completa para fins de conformidade e análise histórica.
- **Indexação (Indexing):** o log é avaliado em relação aos filtros dos Índices definidos. Se ele corresponder a um filtro de índice, ele é então avaliado pelos Filtros de Exclusão (Exclusion Filters) desse índice. Se não for excluído, o log é indexado. A indexação o torna pesquisável no Log Explorer e disponível para análise de padrões e criação de monitores. Logs de DEBUG em produção, por exemplo, podem ser frequentemente excluídos da indexação para controlar custos, mas ainda assim arquivados.

Análise e Visualização (Analysis & Visualization): uma vez indexado, o log pode ser encontrado e analisado no Log Explorer. Os atributos parseados (como orderId) tornam-se Facets, que permitem filtrar e agregar dados de forma interativa. O log aparece correlacionado com traces de APM (se o trace_id estiver presente) e métricas, fornecendo um contexto completo do evento.

Reidratação (Rehydration): meses depois, uma equipe de segurança pode precisar investigar uma anomalia que ocorreu em um período cujos logs não foram indexados. Usando a funcionalidade de Rehydration, eles podem selecionar um intervalo de tempo nos arquivos do S3 e carregar esses logs de volta para o Datadog em um índice temporário, tornando-os totalmente pesquisáveis para a investigação, sem a necessidade de manter dados antigos em um armazenamento “quente” e caro.

Este ciclo de vida completo permite uma abordagem estratégica e econômica para o gerenciamento de logs, garantindo que nenhum dado seja perdido e que o valor de cada log seja maximizado.

Coletando logs de hosts, containers e cloud providers

A capacidade de coletar logs de forma unificada a partir de um ecossistema de TI heterogêneo é um pilar fundamental da plataforma Datadog. O Datadog Agent e as integrações nativas com provedores de nuvem oferecem uma cobertura abrangente, garantindo que nenhum dado de log seja deixado para trás. A seguir, detalhamos os mecanismos de coleta para cada tipo de ambiente.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Coleta em Hosts (Bare Metal ou VMs)

Para servidores físicos ou máquinas virtuais, o **Datadog Agent** é o principal mecanismo de coleta. Uma vez instalado no sistema operacional (Linux, Windows ou macOS), ele pode ser configurado para coletar logs de múltiplas fontes:

- **Tail de Arquivos**: esta é a abordagem mais comum. O Agent é configurado para monitorar um ou mais arquivos de log em tempo real. Por exemplo, em um arquivo conf.d/dotnet.d/conf.yaml, pode-se especificar:

```
logs:
  - type: file
    path: /var/log/my-app/*.log
    service: my-app-service
    source: csharp
```

Figura 3 – Tail de arquivos
Fonte: Elaborado pelo autor (2026)

O Agent lida automaticamente com a rotação de logs e mantém o controle de sua posição em cada arquivo para não perder ou duplicar logs durante reinicializações.

- **Coleta via TCP/UDP**: o Agent pode ser configurado para ouvir em uma porta TCP ou UDP, atuando como um endpoint de syslog para dispositivos de rede ou aplicações que enviam logs através da rede.
- **Fontes Específicas do SO**: em Linux, o Agent pode coletar logs do journald (systemd). Em Windows, ele pode se inscrever em canais do Windows Event Log, o que é crucial para monitorar a saúde do sistema operacional e eventos de segurança.

Coleta em Hosts (Bare Metal ou VMs)

Em ambientes de container, a coleta de logs é projetada para ser dinâmica e automatizada, aproveitando os metadados dos containers.

Docker: a prática recomendada é configurar as aplicações para logar em stdout e stderr. O Datadog Agent, rodando no host, coleta esses streams de todos os

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

containers automaticamente. Ele enriquece os logs com metadados do Docker, como container_name, image_name, e container_id.

Kubernetes: a abordagem padrão é implantar o Datadog Agent como um DaemonSet. Isso garante que uma instância do Agent seja executada em cada nó do cluster. O Agent então coleta os logs de todos os pods em execução naquele nó, geralmente localizados em /var/log/pods/*. A grande vantagem aqui é o Autodiscovery, onde o Agent utiliza anotações nos pods para aplicar configurações de coleta de logs específicas para cada aplicação, sem a necessidade de configuração manual. Por exemplo, uma anotação em um pod pode instruir o Agent a tratar os logs daquele pod como source: csharp e associá-los ao service: payment-api.

```
# Exemplo de anotação em um Deployment do Kubernetes
template:
  metadata:
    annotations:
      ad.datadoghq.com/my-container.logs: '[{"source": "csharp", "service": "payment-api"}]'
```

Figura 4 – Exemplo de anotação em um Deployment do Kubernetes
Fonte: Elaborado pelo autor (2025)

Coleta de Provedores de Nuvem (Cloud Providers)

Para serviços gerenciados que não permitem a instalação de um Agent (como AWS Lambda, S3, ou Azure Functions), o Datadog utiliza integrações baseadas em API e encaminhamento de logs.

Amazon Web Services (AWS): a integração é tipicamente configurada através de um **CloudFormation Stack** fornecido pelo Datadog. Para serviços como AWS Lambda, os logs são escritos no Amazon CloudWatch. A partir daí, uma função Lambda do Datadog (Datadog Forwarder) é subscrita ao grupo de logs do CloudWatch e encaminha os logs para a API do Datadog. Para outros serviços, como logs de acesso do S3 ou logs de fluxo da VPC, os logs são enviados para um bucket S3, e uma notificação de evento aciona o Datadog Forwarder para processá-los.

Microsoft Azure: a coleta é feita através do **Azure Event Hub**. Os logs de diagnóstico de vários serviços do Azure (como App Service, Azure SQL, etc.) podem ser configurados para serem transmitidos para um Event Hub. O Datadog pode então

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

ser configurado para consumir eventos deste Event Hub, seja através de uma Azure Function fornecida pelo Datadog ou através de uma integração direta.

Google Cloud Platform (GCP): a abordagem recomendada é usar um **Google Cloud Pub/Sub Sink**. Os logs do Google Cloud Logging (anteriormente Stackdriver) são exportados para um tópico Pub/Sub. O Datadog então puxa os logs deste tópico Pub/Sub através de uma integração configurada na UI do Datadog.

Essa abordagem de coleta multifacetada garante que, independentemente de onde uma aplicação ou serviço esteja sendo executado, seus logs possam ser centralizados, processados e analisados na plataforma Datadog, fornecendo uma visão verdadeiramente unificada da saúde e do desempenho do sistema.

Logs estruturados vs. não estruturados

A diferença real no troubleshooting

A distinção entre logs estruturados e não estruturados é um dos conceitos mais importantes no gerenciamento de logs moderno. Enquanto os logs não estruturados são essencialmente texto de forma livre, os logs estruturados seguem um formato consistente e previsível, geralmente JSON, que trata cada entrada de log como um conjunto de pares chave-valor. Esta estrutura não é apenas uma questão de formatação; ela transforma os logs de simples mensagens legíveis por humanos em dados ricos e pesquisáveis por máquinas.

A diferença fundamental entre logs estruturados e não estruturados se manifesta de forma mais impactante durante o troubleshooting de um incidente. Em um cenário de alta pressão, a capacidade de filtrar, agregar e analisar logs rapidamente é crucial. É aqui que os logs estruturados oferecem uma vantagem decisiva.

Logs Não Estruturados: A Busca por Agulha no Palheiro

Um log não estruturado típico de uma aplicação .NET pode se parecer com isto:

```
2025-12-08 14:32:15.123 [Error] Failed to process payment for order 789123 for user user@example.com. Reason: Insufficient funds.
```

Figura 5 – Logs Não Estruturados

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Fonte: Elaborado pelo autor (2026)

Para encontrar todos os erros de pagamento, um engenheiro teria que realizar uma busca de texto livre por uma string como "Failed to process payment". Para encontrar os erros de um usuário específico, a busca se torna mais complexa: "Failed to process payment" AND "user@example.com". E se quisermos encontrar todos os erros de “Insufficient funds”? A busca seria por essa string. O problema é que essa abordagem é frágil e ineficiente:

- **Falta de Precisão**: uma busca por "Insufficient funds" pode retornar logs de diferentes contextos, não apenas de pagamentos.
- **Dificuldade de Agregação**: é quase impossível agregar dados de forma confiável. Perguntas como “Qual o ID de pedido mais associado a falhas de pagamento?” ou “Qual a distribuição de motivos de falha?” exigem parsing complexo e propenso a erros em tempo de busca.
- **Performance**: buscas de texto livre em grandes volumes de dados são computacionalmente caras e lentas.

Logs Estruturados: Precisão Cirúrgica com Facets

Agora, considere o mesmo evento logado de forma estruturada:

```
{
  "timestamp": "2025-12-08T14:32:15.123Z",
  "level": "Error",
  "message": "Failed to process payment",
  "orderId": "789123",
  "userEmail": "user@example.com",
  "paymentGateway": "Stripe",
  "failureReason": "insufficient_funds"
}
```

Figura 6 –Logs Estruturados
Fonte: Elaborado pelo autor (2026)

Quando o Datadog ingere este log JSON, ele automaticamente **promove** cada par chave-valor a um **atributo** do log. Esses atributos podem então ser transformados

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

em **Facets**. Um Facet é um atributo indexado que pode ser usado para filtrar, pesquisar e agregar logs de forma eficiente na UI do Datadog. A diferença no troubleshooting é gritante:

- **Buscas Eficientes e Precisas**: em vez de buscas de texto, as consultas são feitas sobre os facets. Para encontrar todos os erros de pagamento, a busca é @level:Error AND @message:"Failed to process payment". Para um usuário específico, @useremail:user@example.com. Para um motivo específico, @failureReason:insufficient_funds. Essas buscas são extremamente rápidas e precisas.
- **Análise e Agregação**: com os facets, é trivial responder a perguntas complexas. No Log Explorer, pode-se agrupar por @failureReason para ver um gráfico com a distribuição de todos os motivos de falha. Pode-se filtrar por @paymentGateway:Stripe e agrupar por @failureReason para ver os principais problemas com um gateway específico.
- **Contexto Rico**: os logs estruturados carregam um contexto rico. O orderId pode ser usado para encontrar todos os outros logs relacionados a esse pedido, e o userEmail para ver a jornada completa daquele usuário.

A tabela abaixo resume as diferenças:

Característica	Logs Não Estruturados	Logs Estruturados
Busca	Texto livre, lenta, imprecisa	Baseada em Facets, rápida, precisa
Filtragem	Limitada a substrings	Granular, por qualquer atributo (Facet)
Agregação	Difícil, propensa a erros	Nativa, poderosa (contagens, médias, etc.)
Visualização	Lista de texto	Gráficos, tabelas, análise de padrões
Correlação	Manual, baseada em texto	Automática com outros logs, traces e métricas

Figura 7 – Logs Estruturados X Logs Não Estruturados
Fonte: Elaborado pelo autor (2026)

Em suma, os logs estruturados transformam o troubleshooting de uma arte reativa e demorada em uma ciência proativa e eficiente, permitindo que os

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

engenheiros identifiquem a causa raiz de um problema em segundos, em vez de horas.

Implementando logs estruturados

A implementação de logs estruturados em uma aplicação .NET é grandemente simplificada pelo uso de bibliotecas de logging modernas, com o Serilog sendo o padrão de fato para esta tarefa. O Serilog foi projetado desde o início com o logging estruturado em mente, tornando a transição de logs de texto para logs ricos em dados uma tarefa simples e intuitiva.

Configurando o Serilog em uma Aplicação .NET

O primeiro passo é adicionar os pacotes NuGet necessários ao projeto. Para uma aplicação asp.net Core que loga para o console em formato JSON, os pacotes essenciais são:

- Serilog.AspNetCore: integra o Serilog ao sistema de logging do asp.net Core.
- Serilog.Sinks.Console: fornece um “sink” (destino) para escrever logs no console.

A configuração é feita no arquivo Program.cs da aplicação:

```
Log.Logger = new LoggerConfiguration()
    .Enrich.FromLogContext()
    .WriteTo.Console(new JsonFormatter())
    .CreateLogger();
```

Figura 8 – Configuração
Fonte: Elaborado pelo autor (2026)

Nesta configuração, .WriteTo.Console(new JsonFormatter()) é a linha chave. Ela instrui o Serilog a enviar todos os logs para o console, formatados como JSON. Quando esta aplicação é executada em um container, o Datadog Agent pode coletar este output JSON diretamente do stdout.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Escrevendo Logs Estruturados

A beleza do Serilog está em suas **Message Templates**. Em vez de usar interpolação de strings, você usa placeholders nomeados na mensagem de log. O Serilog captura o nome e o valor do placeholder como um campo estruturado.

```
        var paymentGateway = "Stripe";
        _logger.LogInformation("Processing payment for OrderId {OrderId} via {Gateway}", request.OrderId, paymentGateway);

        try
        {
            // ... tenta processar o pagamento ...
            throw new InvalidOperationException("Insufficient funds");
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Failed to process payment for OrderId {OrderId}. Reason: {FailureReason}", request.OrderId, "in
sufficient_funds");
            return BadRequest();
```

Figura 9 – Message Templates
Fonte: Elaborado pelo autor (2026)

O log de erro resultante será um JSON contendo os campos OrderId e FailureReason, além de todos os detalhes da exceção, prontos para serem transformados em Facets no Datadog:

```
{
  "level": "Error",
  "message": "Failed to process payment for OrderId \"789123\". Reason: \"insufficient_funds\"",
  "OrderId": "789123",
  "FailureReason": "insufficient_funds",
  "exception": "System.InvalidOperationException: Insufficient funds..."
}
```

Figura 10 – Facets
Fonte: Elaborado pelo autor (2026)

Importante: Note que os placeholders {OrderId} e {FailureReason} na string de formato se tornam chaves no JSON. O @ na frente de um placeholder (ex: {@Request}) diz ao Serilog para serializar o objeto inteiro, em vez de apenas chamar ToString() nele, o que é útil para logar objetos complexos.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Ao adotar esta abordagem, a aplicação começa a emitir logs que são inerentemente pesquisáveis e analisáveis, estabelecendo a fundação para uma observabilidade robusta sem a necessidade de regras de parsing complexas no lado do coletor.

Enriquecimento de logs com contexto de negócio

Logs estruturados são a base, mas seu verdadeiro poder é desbloqueado quando eles são enriquecidos com contexto de negócio. Um log de erro que diz “Falha na conexão com o banco de dados” é útil, mas um log que diz “Falha na conexão com o banco de dados ao tentar processar um pagamento de R$ 5.000 para um cliente do plano Enterprise” é transformador. Este contexto adicional permite que as equipes priorizem problemas, entendam o impacto no negócio e correlacionem eventos técnicos com resultados comerciais.

O Serilog oferece mecanismos poderosos para enriquecer logs de forma consistente em toda a aplicação, principalmente através do LogContext e de middlewares customizados.

Enriquecimento Dinâmico com LogContext

O LogContext permite adicionar propriedades a um “escopo” de log. Todas as mensagens de log escritas dentro desse escopo herdarão automaticamente essas propriedades. Isso é extremamente útil em aplicações web, onde um escopo pode representar uma única requisição HTTP.

Um padrão comum é criar um middleware no asp.net Core que enriquece o contexto do log com informações da requisição, como o ID do usuário autenticado, o ID do tenant (em aplicações multi-tenant) ou outras informações de cabeçalho.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

```
public Task Invoke(HttpContext context)
{
    // Tenta obter o ID do usuário a partir do claim
    var userId = context.User.FindFirst("sub")?.Value;
    // Tenta obter o ID do tenant a partir de um cabeçalho
    var tenantId = context.Request.Headers["X-Tenant-Id"].FirstOrDefault();

    // Adiciona as propriedades ao LogContext. O using garante que as propriedades
    // sejam removidas ao final da requisição.
    using (Serilog.Context.LogContext.PushProperty("UserId", userId ?? "anonymous"))
    using (Serilog.Context.LogContext.PushProperty("TenantId", tenantId ?? "default"))
    {
        return _next(context);
    }
}
```

Figura 11 – LogContex
Fonte: Elaborado pelo autor (2026)

Com este middleware em vigor, cada log gerado durante o processamento de uma requisição HTTP incluirá automaticamente os campos UserId e TenantId. Agora, uma mensagem de erro genérica como “Falha ao acessar o recurso” se torna muito mais acionável:

```
{
  "level": "Error",
  "message": "Failed to access resource",
  "UserId": "auth0|123456789",
  "TenantId": "acme-corp",
  "exception": "System.UnauthorizedAccessException: ..."
}
```

Figura 12 – Falha
Fonte: Elaborado pelo autor (2026)

Isso permite que a equipe de suporte filtre imediatamente todos os erros para um tenant específico (@TenantId:acme-corp) ou investigue todas as atividades de um único usuário (@UserId:auth0|123456789).

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Configurações práticas para produção

A forma como os logs são fisicamente escritos e coletados em um ambiente de produção tem um impacto direto na confiabilidade e no custo da sua estratégia de logging. A escolha entre logar para a saída padrão (stdout), para arquivos, ou usar um sink direto via HTTP depende da arquitetura da sua aplicação e da sua infraestrutura.

Logging para stdout: O Padrão para Ambientes de Container

Em ambientes de container como Docker e Kubernetes, a prática recomendada é configurar a aplicação para escrever seus logs (em formato JSON, como vimos) diretamente para a **saída padrão (stdout) e erro padrão (stderr)**. Esta abordagem é conhecida como o padrão de “log shipping sidecar” ou, mais simplesmente, delega a responsabilidade da coleta de logs para a infraestrutura de orquestração.

- **Simplicidade na Aplicação**: a aplicação não precisa se preocupar com caminhos de arquivo, permissões, ou rotação de logs. Ela simplesmente escreve para o console.
- **Coleta Centralizada**: o Datadog Agent, rodando como um DaemonSet no Kubernetes, é configurado para coletar o stdout/stderr de todos os containers em um nó. Isso centraliza a configuração da coleta e a torna mais gerenciável.
- **Enriquecimento Automático**: o Agent enriquece automaticamente os logs com metadados do container, como pod_name, namespace, image_name, etc., o que é crucial para o troubleshooting em ambientes dinâmicos.

Logging para Arquivos (File Handlers) e Rotação

Para aplicações rodando em máquinas virtuais (VMs) ou servidores bare-metal, logar para arquivos ainda é uma abordagem robusta e comum. Neste cenário, a aplicação escreve para um arquivo no disco, e o Datadog Agent é configurado para fazer o “tail” (monitorar) desse arquivo.

- **Persistência Local**: os logs são persistidos no disco localmente, o que pode ser útil como um backup temporário se a conexão com o Datadog for interrompida.
- **Desacoplamento**: a aplicação e o Agent operam de forma independente. O Agent pode ser reiniciado sem afetar a escrita de logs da aplicação.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Sink Direto para Datadog (Uso Limitado)

O Serilog oferece um sink (Serilog.Sinks.Datadog.Logs) que envia logs diretamente para a API do Datadog via HTTPS. Embora pareça conveniente, esta abordagem tem desvantagens significativas em ambientes de produção:

- **Acoplamento**: a aplicação fica diretamente acoplada à API do Datadog. Se a API estiver indisponível ou lenta, isso pode impactar a performance da aplicação.
- **Falta de Enriquecimento do Agent**: perde-se todo o enriquecimento automático de metadados que o Datadog Agent fornece (tags de host, Docker, Kubernetes).
- **Gerenciamento de Rede**: a aplicação precisa de acesso de rede direto para o endpoint do Datadog, o que pode ser um problema de segurança e configuração.

Para a maioria dos cenários de produção, a combinação de **stdout para containers** e **logging para arquivos com rotação para VMs** é a mais robusta, confiável e recomendada. O sink direto deve ser reservado para casos de uso específicos onde a instalação de um Agent é impossível, como em algumas plataformas serverless de primeira geração ou ambientes muito restritos.

Correlação automática entre logs e traces

Os bastidores da correlação automatica

A capacidade de correlacionar logs e traces é o que eleva a observabilidade de um sistema de um conjunto de sinais isolados para uma narrativa coesa sobre o comportamento da aplicação. Esta correlação permite que um engenheiro navegue de um trace de uma requisição lenta ou com erro diretamente para os logs exatos que foram gerados durante a execução daquela requisição, e vice-versa. Isso reduz drasticamente o tempo médio de resolução (MTTR) de incidentes.

A mágica da correlação entre logs e traces reside na injeção automática de identificadores de contexto do trace nos registros de log. O Datadog .NET Tracer (dd-trace-dotnet) é projetado para fazer isso de forma transparente, integrando-se

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

profundamente com o Common Language Runtime (CLR) e as bibliotecas de logging mais comuns.

O Mecanismo: CLR Profiling API e Injeção Automática

O .NET Tracer do Datadog utiliza a **CLR Profiling API**, uma interface poderosa que permite que um profiler (neste caso, o tracer do Datadog) se anexe a um processo .NET e reescreva o Intermediate Language (IL) dos métodos em tempo de execução (Just-In-Time compilation). É assim que a instrumentação automática funciona: o tracer injeta código no início e no fim de métodos de bibliotecas conhecidas (como ASP.NET Core, HttpClient, clientes de banco de dados) para criar spans de APM.

Quando um span está ativo em uma thread, o tracer armazena seu contexto (que inclui o trace_id e o span_id) em um local acessível naquela thread (como um AsyncLocal<T>).

Para a correlação de logs, o tracer estende essa capacidade. Ao habilitar a injeção de logs (através da variável de ambiente DD_LOGS_INJECTION=true, que é o padrão em versões recentes), o tracer também instrumenta as bibliotecas de logging (Serilog, NLog, log4net).

Quando sua aplicação chama _logger.LogInformation(...), o tracer intercepta essa chamada. Antes que o log seja escrito, ele acessa o contexto do span ativo na thread atual e adiciona os seguintes campos ao registro de log:

- dd.trace_id: o ID do trace atual.
- dd.span_id: o ID do span ativo no momento em que o log foi escrito.
- dd.service: o nome do serviço, conforme configurado pelo DD_SERVICE.
- dd.env: o ambiente, conforme configurado pelo DD_ENV.
- dd.version: a versão da aplicação, conforme configurada pelo DD_VERSION.

Logs aparecendo dentro do trace na UI

Uma vez que os logs são enriquecidos com os IDs de correlação e ingeridos pelo Datadog, a plataforma os conecta visualmente aos seus traces correspondentes, proporcionando uma experiência de troubleshooting unificada e contextual.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Uma vez que os logs são enriquecidos com os IDs de correlação e ingeridos pelo Datadog, a plataforma os conecta visualmente aos seus traces correspondentes, proporcionando uma experiência de troubleshooting unificada e contextual.

A Visualização na UI do Datadog

Ao abrir um trace na seção de APM do Datadog, você verá uma "flame graph" que representa a hierarquia e a duração dos spans naquela requisição. Se a correlação de logs estiver funcionando, você notará duas coisas:

- **Aba "Logs" no Painel do Span**: ao selecionar um span específico no flame graph, o painel de detalhes à direita exibirá várias abas, como "Info", "Metadata", "Metrics", e uma aba **"Logs"**. Esta aba "Logs" mostrará uma lista de todas as mensagens de log que foram geradas durante a execução daquele span específico (ou seja, todos os logs que contêm o span_id daquele span).
- **Marcadores de Log no Flame Graph**: para logs com status de Error ou Fatal, o Datadog exibe um marcador visual diretamente no flame graph, na linha do tempo do span onde o erro ocorreu. Isso permite que você identifique visualmente e instantaneamente qual parte de uma requisição complexa encontrou um problema. Passar o mouse sobre o marcador de erro geralmente mostra um resumo da mensagem de log, e clicar nele leva você diretamente para o log completo.

Navegando de Logs para Traces

A correlação é bidirecional. Ao visualizar logs no Log Explorer, se um log estiver correlacionado a um trace, ele exibirá um link para o trace. Você pode clicar no trace_id ou em um botão "View Trace" para ser levado diretamente ao flame graph completo daquele trace na UI de APM. Isso é útil quando você começa sua investigação a partir de um alerta de log ou de uma busca por uma mensagem de erro específica.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Pipelines de processamento e indexação

A anatomia de um pipeline: ordem importa

Os pipelines de processamento e a configuração de indexação são o coração da capacidade do Datadog de transformar um dilúvio de dados de log brutos em insights estruturados e acionáveis, ao mesmo tempo em que fornecem controle granular sobre os custos. Um pipeline é uma sequência de processadores que enriquecem e transformam os logs à medida que são ingeridos, enquanto os índices determinam quais logs são retidos para análise e por quanto tempo.

Um pipeline de processamento de logs no Datadog é uma sequência ordenada de etapas (processadores) que são aplicadas a um log após sua ingestão. A ordem dessas etapas é crucial, pois o output de um processador se torna o input do próximo. Um pipeline bem estruturado segue uma lógica que progressivamente transforma o log de dados brutos em informação contextualizada. Estrutura de um Pipeline:

Filtro do Pipeline (Opcional): no topo de cada pipeline, há um filtro de consulta. Apenas os logs que correspondem a esta consulta entrarão no pipeline. Isso permite criar pipelines especializados para diferentes fontes de dados (ex: source:nginx, service:payment-api). Se não houver filtro, o pipeline se aplica a todos os logs.

Sequência de Processadores: dentro do pipeline, os processadores são executados em uma ordem de cima para baixo. A ordem típica e mais eficaz segue um padrão de "refinamento progressivo":

- **Parsing (Extração)**: esta é quase sempre a primeira etapa. O objetivo é extrair a estrutura do log. Se o log estiver em JSON, o Datadog faz isso automaticamente. Se for texto plano (ex: logs de acesso do Nginx), um **Processador Grok** é usado para analisar a string e extrair atributos. Por exemplo, uma regra Grok pode extrair http.method, http.url, e http.status_code de uma linha de log de acesso.
- **Remapeamento e Normalização (Remap)**: uma vez que os atributos são extraídos, a próxima etapa é normalizá-los. O **Attribute Remapper** é a ferramenta principal aqui. Ele é usado para: Renomear atributos para seguir uma convenção padrão (ex: renomear http_response_code para

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

http.status_code). Mover atributos para o local correto na estrutura do log. Definir o service ou source de um log com base em um atributo existente.

- **Processamento de Status (Status)**: muitos logs contêm um nível de severidade (ex: INFO, WARN, ERROR). O **Status Remapper** é um processador especializado que mapeia o conteúdo de um atributo de log para o status oficial do log no Datadog. Por exemplo, ele pode ser configurado para que se o atributo log_level contiver a string "Error" ou "Critical", o status do log seja definido como ERROR. Isso é crucial para a visualização (logs de erro aparecem em vermelho) e para a criação de monitores.
- **Categorização e Enriquecimento (Categorization)**: com a estrutura básica e o status definidos, a etapa final é adicionar contexto de negócio ou de domínio. O **Category Processor** pode ser usado para adicionar tags com base em padrões. Por exemplo, se o atributo http.url contiver /api/v1/users, um processador de categoria pode adicionar a tag api_version:v1. Um **Lookup Processor** pode enriquecer o log com informações de uma tabela de referência (ex: mapear um user_id para um user_plan a partir de uma lista carregada).

Por que a Ordem Importa: Um Exemplo

Imagine que você quer definir o status de um log como ERROR se o http.status_code for 500. Se você colocar o Status Remapper *antes* do Processador Grok que extrai o http.status_code, o Status Remapper não encontrará o atributo e não fará nada. O Processador Grok deve vir primeiro para que o atributo http.status_code esteja disponível para os processadores subsequentes.

Da mesma forma, se você quiser remapear um atributo, o parsing que o extrai deve ter acontecido antes. Fluxo lógico de um pipeline bem desenhado:

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Ordem	Etapa	Processador Comum	Propósito
1	**Parsing**	Grok Parser, JSON Parser	Extrair estrutura e atributos do texto bruto.
2	**Remapping**	Attribute Remapper, Date Remapper	Normalizar nomes de atributos, definir data/hora oficial.
3	**Status**	Status Remapper	Definir o nível de severidade do log (INFO, WARN, ERROR).
4	**Categorization**	Category Processor, Lookup Processor	Adicionar contexto de negócio, tags e metadados.

Figura 13 – Fluxo de um pipeline
Fonte: Elaborado pelo autor (2026)

Análise avançada com Facets

Facets são a ponte entre os dados brutos dos logs e a análise interativa e visual no Datadog. Um facet é, essencialmente, um atributo de log que foi indexado para permitir filtragem e agregação rápidas. Enquanto qualquer atributo extraído em um pipeline pode ser pesquisado, transformá-lo em um facet desbloqueia todo o potencial analítico do Log Explorer.

Existem dois tipos de facets:

Facets de String (Qualitativos): usados para atributos com um conjunto finito de valores, como http.method (GET, POST), env (prod, staging), ou user_plan (free, premium). Eles permitem que você filtre por esses valores e veja a contagem de logs para cada valor.

Facets de Medida (Quantitativos): usados para atributos numéricos, como duration_ms, request_size, ou order_value. Eles permitem não apenas filtrar por intervalos (ex: duration_ms:>1000), mas também calcular agregações como média, soma, mínimo, máximo e percentis (p50, p75, p90, p99).

Criando um Facet

Para criar um facet, você navega até a lista de atributos de log no Datadog (geralmente clicando em um atributo em um log no Log Explorer) e seleciona "Create Facet". O Datadog então começa a indexar os valores daquele atributo para análise.

A verdadeira análise avançada começa quando você cria facets a partir dos atributos de negócio que foram adicionados durante o enriquecimento dos logs.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

@**user_plan**: ao criar um facet a partir do user_plan, você pode instantaneamente comparar o comportamento da aplicação para diferentes tipos de clientes. Você pode filtrar por @user_plan:enterprise e ver se eles estão enfrentando mais erros ou maior latência do que os usuários do plano free. Isso ajuda a priorizar correções que impactam os clientes mais valiosos.

@**endpoint**: um facet no endpoint da API (/api/users, /api/payments) permite que você veja rapidamente quais endpoints estão gerando mais logs, mais erros, ou têm a maior latência média. No Log Explorer, você pode agrupar por @endpoint e medir a latência média (duration_ms) para cada um.

@**duration**: um facet de medida no tempo de duração de uma operação (duration_ms) é fundamental para a análise de performance. Você pode usá-lo para:

- Encontrar todos os logs de requisições que levaram mais de 1 segundo: @duration_ms:>1000.
- Visualizar a distribuição de latência ao longo do tempo, incluindo p95 e p99, para entender a experiência dos seus usuários mais afetados.
- Correlacionar alta duração com outros facets. Por exemplo, você pode descobrir que a latência é alta apenas quando o facet @feature_flag:new-algorithm está ativo.

Indexação: retenção e ingestão customizada

A indexação é o processo que torna os logs pesquisáveis e analisáveis no Datadog. No entanto, como a indexação é o principal motor de custos do Log Management, é crucial configurar os índices de forma estratégica. Os Índices de Log no Datadog são a ferramenta para gerenciar quais logs são indexados e por quanto tempo são retidos.

O que é um Índice de Log?

Um índice é como uma tabela em um banco de dados, projetada para um subconjunto específico de logs. Cada índice tem duas configurações principais:

- **Filtro de Ingestão (Ingestion Filter)**: uma consulta que define quais logs devem ser roteados para este índice. Por exemplo, um índice pode ter o

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

filtro service:auth-service, e outro pode ter o filtro env:production AND status:error.

- **Período de Retenção (Retention Period)**: o número de dias que os logs neste índice serão mantidos em armazenamento "quente" e pesquisável. Os períodos de retenção comuns são 3, 7, 15, 30, 60, 90 dias ou mais.

Estratégias de Indexação

Em vez de usar um único índice para todos os logs (o que seria caro e ineficiente), a prática recomendada é criar múltiplos índices com base no valor e nos requisitos de retenção dos logs.

Estratégia Baseada em Ambiente: criar índices separados para cada ambiente.

- **Índice de Produção**: filtro env:production, retenção de 15 dias. Logs críticos para o negócio.
- **Índice de Staging**: filtro env:staging, retenção de 7 dias. Logs para testes pré-produção.
- **Índice de Desenvolvimento**: filtro env:dev, retenção de 3 dias. Logs de depuração de baixo valor.

Estratégia Baseada em Criticidade: reter logs de erro por mais tempo do que logs informativos.

- **Índice de Erros de Produção**: filtro env:production AND status:error, retenção de 30 dias. Permite análises de tendências de erro a longo prazo.
- **Índice Geral de Produção**: filtro env:production, retenção de 7 dias. Captura todos os outros logs de produção para troubleshooting de curto prazo.

Estratégia Baseada em Conformidade: isolar logs que precisam de retenção longa por motivos de segurança ou conformidade.

- **Índice de Auditoria**: filtro source:auditd OR service:iam-service, retenção de 365 dias. Garante que os logs de auditoria estejam disponíveis conforme exigido por regulamentações como PCI-DSS ou SOX.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Filtros de Exclusão (Exclusion Filters)

Dentro de cada índice, você pode refinar ainda mais o que é indexado usando Filtros de Exclusão. Eles são aplicados *depois* que um log corresponde ao filtro de ingestão do índice. Um filtro de exclusão remove logs do índice, mas não da ingestão. Eles ainda podem ser arquivados e usados para gerar métricas.

Esta é uma ferramenta poderosa para otimização de custos. Por exemplo, em um índice de produção com retenção de 15 dias, você pode querer manter todos os logs, exceto os logs de DEBUG.

Índice: env:production (Retenção de 15 dias)

- **Filtro de Exclusão 1**: consulta status:debug, Amostragem de 100%. Isso exclui todos os logs de debug da indexação, economizando custos significativamente.
- **Filtro de Exclusão 2**: consulta source:nginx AND http.status_code:[200 TO 299], Amostragem de 95%. Isso exclui 95% dos logs de sucesso do Nginx, mantendo apenas 5% para análise de tendências, mas indexando 100% dos logs de erro 4xx e 5xx.

Ordem dos Índices

Assim como os processadores em um pipeline, a ordem dos índices importa. Um log flui através da lista de índices de cima para baixo e é roteado para o primeiro índice cujo filtro ele corresponde. Portanto, os índices mais específicos devem vir antes dos mais genéricos.

Ordem Incorreta:

1. Índice Geral (env:production), Retenção 7 dias.
2. Índice de Erros (env:production AND status:error), Retenção 30 dias.

Neste caso, um log de erro de produção corresponderá ao primeiro índice e será retido por apenas 7 dias. O segundo índice nunca será alcançado.

Ordem Correta:

1. Índice de Erros (env:production AND status:error), Retenção 30 dias.
2. Índice Geral (env:production), Retenção 7 dias.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Agora, um log de erro corresponderá ao primeiro índice e será retido por 30 dias. Um log de informação não corresponderá ao primeiro, mas corresponderá ao segundo e será retido por 7 dias. Esta configuração implementa a estratégia de retenção desejada de forma eficaz.

Pipelines vs Índices: decisões de arquitetura

Pipelines e Índices são mecanismos de filtragem e roteamento, mas operam em fases diferentes do ciclo de vida do log e servem a propósitos distintos. Compreender quando usar um em vez do outro é uma decisão de arquitetura fundamental para uma estratégia de logging eficiente e escalável.

Propósitos Fundamentais

Pipelines: o propósito principal de um pipeline é transformar e enriquecer os logs. Eles operam em 100% dos logs ingeridos. A filtragem em um pipeline serve para direcionar um conjunto de transformações a um subconjunto de logs (ex: aplicar um parser Grok apenas a logs com source:nginx).

Índices: o propósito principal de um índice é controlar a retenção e o custo. Eles decidem quais logs serão pesquisáveis e por quanto tempo. A filtragem em um índice é para segregar logs em diferentes "baldes" de armazenamento e custo.

Tabela comparativa de decisão

Característica	Pipeline	Índice
Opera Em	Todos os logs ingeridos	Logs que correspondem ao filtro do índice
Propósito Principal	Transformação e Enriquecimento	Retenção e Controle de Custo
Filtragem Usada Para	Aplicar processadores a um subconjunto de logs	Segregar logs em baldes de retenção
Impacto no Custo	Pode reduzir o custo de ingestão (com `drop`)	Principal ferramenta para controlar o custo de indexação
Pode Descartar Logs?	Sim (processador `drop`)	Não (apenas exclui da indexação)
Ordem Importa?	Sim (ordem dos processadores)	Sim (ordem dos índices)

Figura 14 – Tabela comparativa de decisão
Fonte: Elaborado pelo autor (2026)

A Regra de Ouro: use pipelines para mudar o conteúdo de seus logs. Use índices para decidir o destino e o tempo de vida de seus logs.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Estratégias de amostragem e otimização de custos

Amostragem de traces: head-based vs tail-based

A observabilidade moderna, especialmente em sistemas distribuídos de grande escala, gera um volume de dados (logs, traces, métricas) que pode rapidamente levar a custos proibitivos se não for gerenciado de forma inteligente. A filosofia do Datadog de "Logging without Limits™" e suas estratégias de amostragem de traces são projetadas para enfrentar exatamente este desafio: maximizar a visibilidade e, ao mesmo tempo, manter os custos sob controle.

A amostragem de traces é a prática de selecionar um subconjunto representativo de traces para serem enviados e retidos para análise, em vez de manter 100% deles. Isso é essencial para gerenciar o volume de dados e os custos de APM. Existem duas abordagens principais para a amostragem: head-based e tail-based.

Head-Based Sampling

A amostragem head-based é a forma mais simples e comum de amostragem. A decisão de manter ou descartar um trace é tomada no **início** do trace, no momento em que o primeiro span (o "root span") é criado. Esta decisão é então propagada para todos os outros serviços que participam da mesma transação, geralmente através de cabeçalhos HTTP (como x-datadog-sampling-priority).

Como Funciona: o Datadog Agent ou a biblioteca de tracing (dd-trace-dotnet) aplica uma taxa de amostragem (ex: 10%). Para cada novo trace, um número aleatório é gerado. Se o número estiver dentro da porcentagem definida, o trace é marcado com prioridade "manter" (1) ou "amostrar" (2). Se for marcado para manter, todos os spans subsequentes naquele trace, mesmo em outros microsserviços, herdarão essa decisão e também serão mantidos.

Vantagens:

- **Simplicidade e Baixo Overhead**: é computacionalmente barato e não requer um estado centralizado para tomar a decisão.
- **Consistência**: garante que traces completos sejam mantidos ou descartados, evitando spans órfãos.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Desvantagens:

- **Ponto Cego**: a decisão é tomada antes que o trace seja concluído. Isso significa que um trace que começa normalmente, mas termina em um erro crítico, pode ser descartado se a decisão inicial foi "descartar". Você pode perder visibilidade sobre eventos importantes, mas infrequentes.

Configuração no Datadog: por padrão, o Datadog Agent usa uma amostragem head-based para atingir um alvo de traces por segundo (padrão de 10 TPS). Você pode configurar regras mais granulares na biblioteca de tracing, por exemplo, para amostrar 100% dos traces para um endpoint crítico como /api/payment e 5% para endpoints menos importantes.

Tail-Based Sampling (Amostragem de Cauda)

A amostragem tail-based adota uma abordagem mais inteligente. Em vez de decidir no início, ela coleta **todos** os spans de um trace e toma a decisão de amostragem somente **após** o trace ter sido concluído. Isso permite que a decisão seja baseada no conteúdo completo do trace.

Como Funciona: os spans de um trace são enviados para um coletor (como o Datadog Agent ou um cluster do Observability Pipelines) que os agrupa na memória por trace_id. Uma vez que todos os spans de um trace chegam ou um timeout é atingido, o coletor analisa o trace completo. Ele pode então aplicar regras de decisão, como:

- Manter todos os traces que contêm um span com um status de erro.
- Manter todos os traces cuja duração total excede um certo limiar (ex: > 1 segundo).
- Manter todos os traces que passam por um serviço específico.
- Manter uma porcentagem aleatória dos traces restantes.

Vantagens:

- **Visibilidade Completa de Erros e Latência**: garante que 100% dos traces com erros ou alta latência sejam capturados, que são frequentemente os mais importantes para o troubleshooting.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **Decisões Inteligentes**: permite a criação de regras de amostragem baseadas em qualquer atributo dos spans no trace.

Desvantagens:

- **Maior Complexidade e Custo de Infraestrutura**: requer um componente (o coletor) que possa armazenar temporariamente todos os spans, o que consome mais memória e CPU do que a amostragem head-based.

Implementação no Datadog: o Datadog implementa uma forma de amostragem tail-based através dos **Error and Rare Samplers** no Agent e, de forma mais poderosa, com o **Datadog Observability Pipelines**. O Observability Pipelines pode ser implantado como um gateway que recebe todos os traces, aplica regras de amostragem tail-based e, em seguida, encaminha apenas os traces selecionados para o backend do Datadog, proporcionando o máximo de visibilidade com controle de custos.

Característica	Head-Based Sampling	Tail-Based Sampling
Ponto de Decisão	Início do trace (root span)	Fim do trace (após todos os spans serem coletados)
Critério de Decisão	Aleatório (probabilístico)	Conteúdo do trace (erros, latência, atributos)
Visibilidade de Erros	Pode perder erros se o trace for descartado no início	Garante a captura de 100% dos traces com erro
Overhead	Baixo	Alto (requer buffer para armazenar spans)
Caso de Uso Ideal	Ambientes com tráfego uniforme e onde uma amostragem estatística é suficiente.	Ambientes onde a captura de todos os erros e anomalias de latência é crítica.

Figura 15 – Head-Based vs Tail-Based Sampling
Fonte: Elaborado pelo autor (2026)

Filtros de exclusão (exclusion filters)

Os filtros de exclusão são uma das ferramentas mais poderosas e diretas para otimizar os custos de Log Management no Datadog. Eles operam no nível do Índice e permitem que você evite a indexação de logs de baixo valor, sem deixar de ingeri-los, arquivá-los e gerar métricas a partir deles. Isso se alinha perfeitamente com a filosofia "Logging without Limits™".

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Como Funcionam os Filtros de Exclusão?

Quando um log chega a um índice (após passar pelos pipelines de processamento e corresponder ao filtro do índice), ele é avaliado em relação à lista de filtros de exclusão daquele índice, em ordem sequencial. O log é processado pelo primeiro filtro de exclusão que ele corresponde. Uma vez que um log corresponde a um filtro, ele não é avaliado pelos filtros subsequentes.

Cada filtro de exclusão consiste em três partes:

1. Consulta (Query): uma consulta de log do Datadog que define o subconjunto de logs ao qual o filtro se aplica. Por exemplo, status:info AND source:nginx.
2. Tipo de Exclusão: define a lógica de exclusão:
   - Exclude all logs matching the query: exclui 100% dos logs que correspondem à consulta.
   - Sample a percentage of logs: exclui uma porcentagem dos logs correspondentes, permitindo manter uma amostra estatística.
3. Ativo/Inativo: um toggle para habilitar ou desabilitar o filtro sem precisar excluí-lo.

Estratégias Comuns de Filtros de Exclusão

1. **Excluir Logs de Health Check**: health checks de load balancers ou do Kubernetes podem gerar um volume massivo de logs de sucesso repetitivos. Eles são essenciais para a operação, mas raramente precisam ser analisados individualmente.
   - **Consulta**: http.url_path:/healthz AND http.status_code:200
   - **Ação**: excluir 100% dos logs.
   - **Impacto**: redução massiva de custos com impacto zero no troubleshooting de problemas reais.
2. **Amostrar Logs de Sucesso (2xx):** logs de requisições HTTP bem-sucedidas (status 2xx) são úteis para entender o tráfego, mas você raramente precisa de todos eles. Manter uma pequena amostra é geralmente suficiente.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **Consulta**: source:nginx AND http.status_code:[200 TO 299]
- **Ação**: excluir 95% dos logs (mantendo 5%).
- **Impacto**: grande economia de custos, mantendo a capacidade de analisar tendências de tráfego e performance em logs de sucesso.

3. **Excluir Logs de Debug em Produção**: logs com nível DEBUG são vitais durante o desenvolvimento, mas em produção, seu volume pode ser esmagador. A estratégia aqui é excluí-los por padrão, mas ter a capacidade de reativá-los rapidamente durante um incidente.
   - **Consulta**: status:debug
   - **Ação**: excluir 100% dos logs.
   - **Impacto**: enorme economia de custos. Durante um incidente, este filtro pode ser temporariamente desativado na UI do Datadog para obter visibilidade total, e reativado depois.

Amostragem de logs de debug: reduzir 90% sem perder visibilidade

Logs de DEBUG representam um dilema clássico na observabilidade. Eles contêm informações de diagnóstico incrivelmente detalhadas, que são inestimáveis durante a depuração de um problema complexo. No entanto, seu volume em um ambiente de produção é tão alto que indexá-los continuamente é financeiramente inviável para a maioria das organizações. A solução não é parar de gerar logs de debug, mas sim gerenciá-los de forma inteligente.

A Estratégia: Excluir por Padrão, Habilitar sob Demanda

Continue Gerando Logs de Debug no Código: a aplicação .NET deve continuar a gerar logs de debug detalhados. Use o nível Log.Debug() do Serilog generosamente em torno de lógica complexa, chamadas de I/O e pontos de decisão importantes. O custo de gerar a string de log na memória é insignificante.

Exclua 100% da Indexação por Padrão: no seu principal índice de produção no Datadog, crie um filtro de exclusão com a consulta status:debug e configure-o para excluir 100% dos logs correspondentes. Isso garante que, em operação normal, você não pague para indexar nenhum log de debug.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Arquive Todos os Logs: certifique-se de que sua configuração de arquivamento esteja capturando *todos* os logs ingeridos, incluindo os de debug que foram excluídos da indexação. Eles serão enviados para o seu cold storage (S3, etc.).

Habilite a Indexação Durante Incidentes: quando um incidente ocorre e a equipe de engenharia precisa de visibilidade total, um usuário com as permissões adequadas no Datadog pode, com um único clique, desativar o filtro de exclusão de debug. Imediatamente, os novos logs de debug começarão a fluir para o índice e se tornarão pesquisáveis. A equipe pode então analisar o comportamento detalhado da aplicação para encontrar a causa raiz.

Reative a Exclusão Após o Incidente: uma vez que o incidente seja resolvido, o filtro de exclusão é reativado, e o custo de indexação volta ao normal.

Arquivamento “cold storage” em S3 e reidratação

O arquivamento de logs é um componente essencial de uma estratégia de gerenciamento de logs madura, servindo a dois propósitos principais: conformidade regulatória e análise histórica de baixo custo. O Datadog se integra perfeitamente com os principais provedores de armazenamento em nuvem, como Amazon S3, Google Cloud Storage e Azure Blob Storage, para fornecer uma solução de arquivamento robusta.

Configurando o Arquivamento para Amazon S3

O processo de configuração do arquivamento para um bucket S3 envolve algumas etapas:

Criação do Bucket S3: crie um bucket S3 na sua conta AWS. É uma boa prática criar um bucket dedicado para os logs do Datadog e configurar políticas de ciclo de vida para mover os logs para classes de armazenamento mais baratas (como S3 Glacier) após um certo período.

Configuração de Permissões: no AWS IAM, crie uma política que conceda à role de integração do Datadog as permissões necessárias para escrever objetos no bucket (s3:PutObject). Se você planeja usar a reidratação, você também precisará de permissões de leitura (s3:GetObject, s3:ListBucket).

Configuração no Datadog: na UI do Datadog, na seção de Log Management, navegue para a configuração de Archives. Crie um novo archive, selecione sua conta

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

AWS e a role configurada, e forneça o nome do bucket S3 e um prefixo opcional (como datadog-logs/).

Filtro de Arquivamento: você pode definir uma consulta para especificar quais logs arquivar. A prática recomendada e padrão é usar a consulta *, que significa **arquivar todos os logs ingeridos**, independentemente de terem sido indexados ou não. Isso garante que você tenha um registro completo de todos os eventos para fins de auditoria e análise futura.

Uma vez configurado, o Datadog começará a agrupar os logs ingeridos e a enviá-los para o seu bucket S3 em intervalos regulares, comprimidos em formato JSON.

Reidratação de Logs (Rehydration)

A reidratação é o processo de carregar logs do seu cold storage (S3) de volta para o Datadog para torná-los pesquisáveis. Este é um recurso pago, sob demanda, ideal para investigações forenses, auditorias ou análises de tendências de longo prazo que não justificariam a retenção de dados em armazenamento "quente" (indexado).

Iniciar a Reidratação: na UI do Datadog, navegue até a seção de reidratação. Você selecionará o archive do qual deseja reidratar.

Definir o Escopo: você especificará um **intervalo de tempo** preciso (ex: de 8 de dezembro de 2025, 14:00 a 14:15) e uma **consulta** para limitar os logs a serem reidratados (ex: service:payment-api AND env:production). Isso é crucial para controlar os custos, pois a reidratação é cobrada com base no volume de dados escaneados no S3.

Criação do Índice Temporário: o Datadog iniciará um job que escaneia os arquivos correspondentes no seu bucket S3, extrai os logs que correspondem à sua consulta e ao intervalo de tempo, e os ingere em um novo índice temporário na sua conta. Este índice terá um período de retenção definido por você (ex: 3 dias).

Análise: uma vez que a reidratação esteja completa, os logs aparecerão no Log Explorer como qualquer outro log indexado. Você pode pesquisá-los, criar visualizações e analisá-los usando todo o poder da plataforma. Após o período de retenção do índice de reidratação, os logs são removidos do armazenamento quente do Datadog, mas permanecem intactos no seu bucket S3.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Archive Search vs. Rehydration

Recentemente, o Datadog introduziu o Archive Search, que é uma alternativa à reidratação. Em vez de carregar os dados em um índice, o Archive Search permite que você execute consultas diretamente nos seus arquivos no S3. É geralmente mais rápido para buscas exploratórias, pois começa a retornar resultados em streaming, mas pode ser menos performático para agregações complexas em comparação com um índice totalmente reidratado. A escolha entre os dois depende do caso de uso: use o Archive Search para buscas rápidas e a reidratação para análises profundas e complexas.

Como projetar uma estratégia sustentável de custo

Projetar uma estratégia de custo sustentável para observabilidade não é sobre gastar menos, mas sim sobre gastar de forma inteligente. Trata-se de alinhar os custos de monitoramento com o valor de negócio dos dados, garantindo que você tenha a visibilidade de que precisa sem estourar o orçamento. Uma estratégia sustentável combina todas as técnicas que discutimos em uma abordagem coesa.

Os Pilares de uma Estratégia de Custo Sustentável

Coleta Abrangente, Indexação Seletiva: este é o pilar central. Configure suas aplicações e o Datadog Agent para coletar 100% dos logs e traces. A decisão de não coletar deve ser a exceção. A verdadeira otimização de custos acontece na camada de indexação e retenção.

Logs Estruturados Primeiro: invista tempo para garantir que suas aplicações gerem logs estruturados em JSON. Isso economiza custos a longo prazo, eliminando a necessidade de regras de parsing Grok complexas e frágeis, e torna a análise muito mais eficiente. O custo de CPU para o parsing no pipeline do Datadog, embora pequeno por log, pode se acumular em grandes volumes.

Estratégia de Indexação Multi-Nível: não use um único índice para tudo. Crie múltiplos índices com diferentes períodos de retenção baseados na criticidade e nos requisitos de conformidade.

- **Índice de Erros (30 dias)**: status:error - para análise de tendências de erro.
- **Índice de Segurança (365 dias)**: source:auditd - para conformidade.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **Índice de Produção Geral (7 dias)**: env:production - para troubleshooting do dia-a-dia.
- **Índice de Desenvolvimento (3 dias)**: env:dev - para desenvolvimento ativo.

Uso Agressivo de Filtros de Exclusão: Dentro de seus índices, seja agressivo ao excluir o ruído.

- Exclua 100% dos logs de health check.
- Exclua 100% dos logs de debug por padrão.
- Amostre agressivamente logs de sucesso de alto volume (ex: exclua 90-99% dos logs de status 2xx de CDNs ou load balancers).

Métricas a Partir de Logs: para logs de alto volume e baixa cardinalidade (como logs de acesso de CDN), em vez de indexá-los, gere métricas a partir deles no pipeline de ingestão. Você pode criar uma métrica para contar o número de requisições, com tags para status_code, country e browser. Isso lhe dá a capacidade de criar dashboards e alertas sobre as tendências, a uma fração do custo de indexar os logs brutos.

- **Amostragem Inteligente de Traces**: combine amostragem head-based e tail-based. Use a amostragem head-based por padrão para controlar o volume geral, mas use o Observability Pipelines ou os samplers do Agent para garantir que 100% dos traces com erros ou alta latência sejam capturados.
- **Revisão e Iteração Contínuas**: uma estratégia de custos não é algo que se define uma vez e se esquece. Reúna-se trimestralmente com as equipes de engenharia e finanças para revisar os dashboards de uso do Datadog. Identifique os serviços ou tipos de log mais caros e pergunte: "Ainda estamos obtendo valor com a indexação desses dados? Podemos reduzir a retenção? Podemos amostrar mais agressivamente?".

Uma abordagem prática é pensar em termos de "orçamentos de observabilidade" por equipe ou serviço. Uma equipe que gerencia um serviço de pagamento crítico pode ter um orçamento de indexação maior do que uma equipe que gerencia um serviço interno de baixa criticidade. Isso incentiva as equipes a serem

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

donas de seus próprios custos de observabilidade e a tomar decisões informadas sobre o que logar e indexar.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, consolidamos os conceitos centrais de Log Management em plataformas cloud-native. Entendemos a diferença prática entre logs estruturados e não estruturados e como isso impacta diretamente o troubleshooting e a eficiência operacional. Aprendemos o ciclo de vida completo de um log no Datadog — ingestão, processamento, roteamento, indexação e reidratação e compreendemos a construção de pipelines e a importância da ordem das transformações para garantir parsing correto e enriquecimento contextual.

Além disso, exploramos a criação e uso estratégico de facets para análise avançada e estudamos a correlação automática entre logs e traces como mecanismo para redução de MTTR. Por fim, analisamos estratégias de indexação, retenção e amostragem como pilares de otimização de custo em ambientes de grande escala.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

REFERÊNCIAS

DATADOG. **Efficiently Retrieve Old Logs with Log Rehydration™**. Disponível em: https://www.datadoghq.com/blog/efficient-log-rehydration-with-datadog/. Acesso em: 09 jan. 2026.

DATADOG. **Log Pipelines – Documentation**. Disponível em: https://docs.datadoghq.com/logs/log_configuration/pipelines/. Acesso em: 09 jan. 2026.

DATADOG. **Logging Without Limits™**. Datadog Blog, 2023. Disponível em: https://www.datadoghq.com/blog/logging-without-limits/. Acesso em: 09 jan. 2026.

MICROSOFT. **.NET Logging Overview**. Disponível em: https://learn.microsoft.com/dotnet/core/extensions/logging. Acesso em: 09 jan. 2026.

SERILOG. **Serilog Documentation**. Disponível em: https://serilog.net. Acesso em: 09 jan. 2026.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

PALAVRAS-CHAVE

Log Management, Correlação Log–Trace, Pipelines & Indexação

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com



PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com